

CSCE 748 COMPUTATIONAL PHOTOGRAPHY · MODULE 02 · LECTURE NOTES

Image Processing Foundations

The operations everything else is built from.

MODULE OUTCOMES

- Implement convolution and explain separability.
- Explain the frequency-domain view and when it helps.
- Build and use Gaussian and Laplacian pyramids.
- Implement an edge-aware filter and explain why it is needed.
- Explain aliasing in images and how to avoid it.

1 Convolution

$$(f * g)[x, y] = \sum_i \sum_j f[i, j] \cdot g[x-i, y-j]$$

Slide a kernel over the image, multiply element-wise, and sum. Nearly every operation in this course is either a convolution or built from several.

Property	Statement	Why it matters
Commutative, associative	$f * g = g * f$ and $(f * g) * h = f * (g * h)$.	Successive blurs compose into a single equivalent kernel, so a chain of filters can be collapsed.
Separability	If $g(x, y) = g_x(x) \cdot g_y(y)$, the 2D convolution is two 1D convolutions.	An $n \times n$ kernel costs n^2 multiplies per pixel; two n-tap passes cost $2n$. For a 31×31 Gaussian that is 961 against 62 — a factor of fifteen. The single most important practical fact in this module.
Linearity	$f * (ag + bh) = a(f * g) + b(f * h)$.	Filters can be combined and decomposed algebraically.

Kernel	Effect	Notes
Box	Uniform average over a window.	Cheapest, and its frequency response oscillates — which produces visible artefacts on structured content.
Gaussian	Distance-weighted average.	Separable, no ringing, and the only kernel whose repeated application stays Gaussian. The default for good reason.
Sobel, Prewitt	Approximate first derivative.	Gradient magnitude gives edge strength and the ratio gives direction.

Kernel	Effect	Notes
Laplacian	Approximate second derivative.	Zero crossings mark edges. Sensitive to noise, hence the next row.
Laplacian of Gaussian	Blur, then second derivative.	Scale-selective — the Gaussian's width selects which scale of edge is detected. The basis of blob detection.
Unsharp mask	$\text{image} + k \cdot (\text{image} - \text{blur})$.	Sharpening is adding back high frequencies that survived, not recovering detail that was lost. Worth internalising before Module 06, where the distinction between amplification and recovery becomes the whole subject.

THE FREQUENCY-DOMAIN VIEW

Convolution in the spatial domain is multiplication in the frequency domain. So every filter can be described by a frequency response — what it does to each spatial frequency.

This makes filter behaviour predictable. A Gaussian blur is a low-pass filter with a smooth rolloff. A Laplacian is a high-pass. Unsharp masking boosts high frequencies. These are statements about the response rather than metaphors.

And it explains the box filter's artefacts. A box in space is a sinc in frequency, which oscillates and has negative lobes — so a box blur does not monotonically attenuate high frequencies and can *amplify* some of them. This is why box blur looks wrong on structured content and Gaussian blur does not.

It also makes large convolutions cheap: forward FFT, multiply, inverse FFT, at $O(n \log n)$ in the image size and independent of kernel size. **This is how the deconvolution of Module 06 is computed,** and it is why a 200-pixel-wide blur kernel is no more expensive than a 5-pixel one.

2 Pyramids

	Gaussian pyramid	Laplacian pyramid
Construction	Repeatedly blur and downsample by a factor of two.	Each level is the difference between a Gaussian level and the upsampled version of the next coarser one.
Each level is	A successively lower-frequency version of the image.	One octave of spatial frequency — a band-pass decomposition.
Invertible?	No — information is discarded at each level.	Yes. Summing the levels (with the coarsest residual) recovers the original exactly, which is what makes it useful for blending (Module 07).
Used for	Coarse-to-fine search, mipmaps, scale-space analysis.	Blending, compression, and detail manipulation — amplifying one band sharpens at that scale only.

Blur before downsampling, always, or the result aliases (§3). This is the most common pyramid bug and it produces artefacts that persist through every subsequent level.

COARSE-TO-FINE IS THE STANDARD SEARCH STRATEGY

Searching for an alignment, a flow field, or a disparity at full resolution is both expensive and prone to local optima — the objective function has many shallow minima from repeated texture.

Solve at the coarsest pyramid level first. The image is small, so the search space is tiny; and the aggressive blurring has removed the high-frequency structure that created the spurious minima, so the objective is smooth and the global optimum is easy to find.

Then refine at each successively finer level, using the previous level's answer (scaled up) as the starting point. Each refinement is a small local search in a neighbourhood already known to be correct.

This is how image alignment (Module 07), optical flow, and stereo matching all work, and it is among the most reusable ideas in the course — it appears any time a search over a large displacement is needed.

3 Edge-aware filtering

A GAUSSIAN BLUR DOES NOT KNOW WHAT AN EDGE IS

Gaussian weights depend only on spatial distance. A pixel ten pixels away contributes the same amount whether it belongs to the same surface or to a completely different object across a strong boundary.

So blurring to suppress noise necessarily blurs edges too, and no choice of σ separates the two — noise and edges occupy overlapping frequency bands.

The bilateral filter adds a second factor to the weight: similarity in *value* as well as proximity in *space*. Pixels that are nearby and similar in intensity contribute strongly; pixels that are nearby but very different contribute almost nothing.

So contributions do not cross strong edges, and the filter smooths within regions while leaving boundaries intact. **The conceptual step is small** — it is still a normalised weighted average, with one extra term in the weight — and the consequences are large.

$$w(p, q) = G_{\sigma_s}(\|p - q\|) \cdot G_{\sigma_r}(|I(p) - I(q)|)$$

where σ_s sets the spatial neighbourhood and σ_r sets what counts as similar in value. **The range term makes the filter non-linear and signal-dependent**, which means it cannot be evaluated with an FFT and a naive implementation is slow — hence the fast approximations below.

- **Bilateral filter.** The original formulation. Naively $O(\text{window size})$ per pixel; the **bilateral grid** and related techniques make it fast by treating it as a linear operation in a higher-dimensional space.
- **Joint (cross) bilateral filter.** Take the *range* weights from a different image than the one being filtered. **This is more powerful than it sounds:** a noisy ambient-light photograph can be filtered using the edges from a clean flash photograph of the same scene, giving noise reduction that preserves detail the noisy image alone could not have supported.
- **Guided filter.** Linear time regardless of window size, often better behaved near edges than the bilateral filter, and the usual modern choice. Also accepts a separate guide image.
- **Anisotropic diffusion.** The partial differential equation formulation of the same idea — diffuse heat through the image, with conductivity reduced across edges.
- **Base/detail decomposition.** Edge-aware filter to get a 'base' layer, divide the original by it to get a 'detail' layer, and then process the two differently. **This is the foundation of local tone mapping** (Module 04) and of a great deal of photographic retouching.

BLUR BEFORE YOU DOWNSAMPLE

Downsampling without first low-pass filtering folds high frequencies down into low ones — the same Nyquist argument as CSCE 641 Module 09, arriving from the image-processing side.

The symptoms are moiré patterns on fine repeating detail — fabric, brickwork, distant fences — **and jagged edges** that shimmer distractingly when the image or the camera moves.

So always low-pass before decimating, with the cutoff matched to the new sampling rate. Halving the resolution means removing everything above half the original Nyquist frequency first.

And this applies to the sensor itself. The colour filter array samples the image, so scene detail finer than the pixel pitch aliases. **This is why cameras have historically included an optical low-pass filter in front of the sensor**, and why removing it — as many modern cameras do — is a deliberate trade of sharpness against moiré.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Szeliski — Computer Vision, Chapter 3 (free)	Linear filtering, pyramids, and edge-aware filters, with the frequency analysis. The reference for this module.
CMU 15-463 — image processing and pyramids lectures (free)	Convolution, pyramids, and the assignment that builds them.
Paris et al. — Bilateral Filtering: Theory and Applications (free)	The definitive survey of §3, including the fast approximations and the joint bilateral applications.
He, Sun & Tang — Guided Image Filtering (free)	The guided filter. Short paper, linear time, and a few lines of code.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement 2D convolution directly and with a separable pass. Verify they agree and report the speedup for kernel sizes from 3 to 51.
- 2 Implement convolution via FFT and find the kernel size at which it overtakes the direct separable method.
- 3 Plot the frequency response of a box filter and a Gaussian of the same width. Identify the box filter's negative lobes.
- 4 Apply both to an image of a brick wall and explain the difference from the frequency plots.

- 5 Implement unsharp masking and demonstrate that it cannot recover detail that has been blurred away — blur an image, sharpen it, and compare against the original.
- 6 Build Gaussian and Laplacian pyramids. Reconstruct the image from the Laplacian pyramid and confirm the error is at floating-point level.
- 7 Downsample an image with and without pre-blurring. Photograph or render something with fine repeating detail to make the aliasing obvious.
- 8 Implement the bilateral filter. Compare against a Gaussian of the same spatial σ on a noisy image with strong edges.
- 9 Implement a joint bilateral filter and use a flash image to denoise a no-flash image of the same scene.
- 10 Implement base/detail decomposition and amplify the detail layer. This is the core of local tone mapping and of most 'clarity' sliders.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Write the convolution sum and state three properties.
- 2 What is separability and what does it save for a 31×31 kernel?
- 3 What does convolution correspond to in the frequency domain, and what does that explain?
- 4 Why does a box filter produce artefacts a Gaussian does not?
- 5 Why is sharpening not detail recovery?
- 6 Distinguish Gaussian and Laplacian pyramids, and say which is invertible.
- 7 Describe coarse-to-fine search and say what problem it solves.
- 8 Why can a Gaussian blur not preserve edges, and what does the bilateral filter add?
- 9 Name four edge-aware filters and one application of the joint bilateral filter.
- 10 Why must you blur before downsampling, and what does that have to do with optical low-pass filters?